

→ subqueries are very difficult to comprehend.

→ CTE

With CTE

→ having using CTE we can use it multiple times, they are separated for readability

→ but it is not a actual table.

CTE → common table expression

↓

is a reference. within select, insert, update,
(or) delete query

CTE should be followed by select.

↓

scope is still at query level.

Recursive CTE → repetitively.

↓

example

we can have each day payments calling the
recursive CTE everyday

need to revise

Temporary tables (existing till the table is dropped [or] the session exists).

→ creates a physical table

↓
scope is shifted from query level to session level.

→ not in DB but only in memory.

→ These tables can't be shared, only on local memory. but can't be shared over cloud.

⇒

VIEWS (as a virtual table) (permanently).

↓
tables can be shared

→ every time a view is called will it create a view again ??

↓
only the ~~base~~ print exist but is executed on-demand

→ abstraction → protection of data.

↓
only the columns created in view are the accessed using a query by anyone.

↓
can be based on ~~role~~ role based access.

"views can be joined" → can be parallelly execution.

Stored procedures:
↓
for reusability

(physical scheme) -

authentication

input parameter

output parameters.

Call

Can be used for frontend access

@ total → output parameter.

↓
Can be reverse the calls stacked instead of output parameters.

Dynamic querying

10 - 5 Subquery

2, CTE

1, Temp

2, View

} → stored procedures

↓
using dynamic query.

7. joins → CTE, temp, views (a)

both queries.